Simplicial Complex Multiset Embedders

Algorithms 1 and 2 — Strategy and Implementation (v2)

June 12, 2026

Contents

1 What We Are Trying to Do

The input is an unordered multiset $M = \{\mathbf{v}_0, \ldots, \mathbf{v}_{n-1}\}$ of n vectors in $\mathbb{R}^k$. Concretely we represent M as a $k \times n$ matrix whose columns are the vectors (column j = vector $\mathbf{v}_j$, row i = coordinate i). Column order is arbitrary.

We want a map f from such multisets to some $\mathbb{R}^N$ that is:

- **Multiset-invariant** — the output does not depend on how the columns are ordered.
- **Injective** — distinct multisets give distinct outputs (f is an *embedder*).
- **Continuous** (C^0) — small changes in the vectors produce small changes in $f(M)$.
- **Positively homogeneous of degree 1** — $f(\lambda M) = \lambda f(M)$ for all $\lambda > 0$.

Multiset invariance and continuity are in tension: any map that achieves invariance by sorting will have kinks wherever the sorted order changes. The strategy below resolves this tension elegantly.

2 The Strategy

Step A — Write M in a natural basis

Label each of the nk matrix entries by e_i^j (column j, row i). Let $X[j][i]$ denote the entry in column j, row i. Then

$$M = \sum_{j,i} X[j][i]\, e_i^j,$$

where e_i^j is the $k \times n$ matrix that is 1 at (i, j) and 0 elsewhere. This is the expansion of M in the *standard Eji basis*.

Step B — Peel off the fully symmetric parts

A *fully symmetric* basis element at row i is $\sum_j e_i^j$: it places the same coefficient on every vector at coordinate i, so it is unaffected by any permutation of the columns. The smallest coefficient that any vector has at row i is $m_i = \min_j X[j][i]$. Factoring:

$$M \;=\; \underbrace{\sum_i m_i \left(\sum_j e_i^j\right)}_{\text{symmetric part}} \;+\; \underbrace{\text{balance}}_{\text{all entries} \geq 0}.$$

The symmetric part is already permutation-invariant; its k scalar coefficients $m_0, \ldots, m_{k-1}$ can be stored directly as anchor values.

The balance $M - \sum_i m_i(\sum_j e_i^j)$ has all non-negative entries and contains the non-trivial, non-symmetric information in M.

Step C — Re-express the balance as a positive combination of nested basis elements

Within row i, order the n vectors descending by their (shifted) value. Let $\sigma_i(0), \ldots, \sigma_i(n-1)$ be this ordering. The gaps $\delta_r^{(i)} = X[\sigma_i(r)][i] - X[\sigma_i(r+1)][i] \geq 0$ are the drops between consecutive ranks.

By Abel summation (the discrete analogue of integration by parts), the balance at row i becomes:

$$\text{balance at row } i \;=\; \sum_{r=0}^{n-2} \delta_r^{(i)} \cdot V_r^{(i)},$$

where the *prefix basis element* $V_r^{(i)} = \left\{e_i^{\sigma_i(0)}, \ldots, e_i^{\sigma_i(r)}\right\}$ is the set of the top-$(r{+}1)$ vectors by row-i value.

This is another change of basis: from individual Ejis with signed coefficients to nested prefix sets with non-negative gap coefficients. The structure of which vectors appear in $V_r^{(i)}$ — which j's dominate row i — is the non-symmetric information not yet discarded.

Step D — Merge, re-sort, and apply a second Abel summation

Collect all $m = k(n-1)$ gap–prefix-element pairs from all rows into one list and sort descending by gap: $\delta_{(0)} \geq \cdots \geq \delta_{(m-1)}$ with corresponding basis elements $V_{(0)}, \ldots, V_{(m-1)}$.

Apply a second Abel summation to obtain *cumulative* basis elements and weighted-difference coefficients:

$$\alpha_s \;=\; (s+1)\big(\delta_{(s)} - \delta_{(s+1)}\big), \qquad \delta_{(m)} := 0,$$
$$L_s \;=\; V_{(0)} + V_{(1)} + \cdots + V_{(s)},$$

where L_s is the *Eji_LinComb*: the $k \times n$ matrix whose (i,j) entry counts how many of the first $s+1$ prefix elements contain e_i^j. Its *index* $s+1$ records how many prefix elements have been accumulated.

The mathematical identity $\sum_s \delta_{(s)} V_{(s)} = \sum_s \alpha_s \cdot (L_s/(s+1))$ confirms this preserves all information. The normalised basis element $L_s/(s+1)$ is the *average* of the first $s+1$ prefix elements. The $(s+1)$ weight in α_s distributes contributions more evenly and ensures that $\sum_s \alpha_s = \sum_s \delta_{(s)}$ (the total weight is preserved).

Crucially, L_s captures *which prefix elements came first* in the global sort: two different orderings of the same set of prefix elements produce different L_s sequences.

Step E — Achieve permutation invariance by canonicalising

The j-indices in L_s still reflect the arbitrary labelling of columns. The *canonical form* $\widetilde{L_s}$ is obtained by sorting the columns of L_s's count matrix lexicographically — this removes the column labelling while retaining the multiset structure. Two inputs related by a column permutation produce identical $\widetilde{L_s}$ sequences and hence identical outputs.

Step F — Encode in $\mathbb{R}^{2m+1}$ and assemble

Map each canonical L_s to a pseudorandom point $p_s = h(\widetilde{L_s}) \in [0,1]^{2m+1}$ via MD5 hashing of its count matrix and index. The main body of the output is $\sum_s \alpha_s p_s$.

> **Why $\mathbb{R}^{2m+1}$ is the right dimension.** All $\alpha_s \geq 0$, so $\sum_s \alpha_s p_s$ is a point in the *positive cone* generated by $\{p_0, \ldots, p_{m-1}\}$, a cone of dimension m. Two m-dimensional positive cones in $\mathbb{R}^{2m+1}$ are *generically disjoint* by dimension counting: $m + m = 2m < 2m+1$. Therefore a single point in $\mathbb{R}^{2m+1}$ encodes both: (a) which cone it lies in (i.e. which canonical $\{L_s\}$, i.e. which basis elements), and (b) the coefficients $\{\alpha_s\}$ within that cone.
> This is also why the algorithm is C^0: when two values in the same row are equal, the gap at that position is 0 and so $\alpha_s = 0$. The point $\sum \alpha_s p_s$ then lies on a shared *face* of two adjacent cones rather than in either interior. That shared face is the geometric expression of the continuity: the change in which cone we are in happens exactly at the moment when the coefficient of the transitioning basis element is zero, so the output is unaffected.

The full output is:

$$f(M) \;=\; \Big[\, \underbrace{m_0, \ldots, m_{k-1}}_{k \text{ row minima}},\; \underbrace{\textstyle\sum_{s=0}^{m-1} \alpha_s p_s}_{2m+1 \text{ reals}} \,\Big] \;\in\; \mathbb{R}^{k+2m+1}.$$

With $m = k(n-1)$: total size $= k + 2k(n-1) + 1 = 2nk + 1 - k$.

3 Two Algorithms, One Strategy

Both algorithms follow Steps A–F. They differ only in **Step B**: how many symmetric parts are peeled off, and what "fully symmetric" means.

Algorithm 2 (per-row). Factors out the k per-row symmetric elements $\sum_j e_i^j$, one per coordinate. Stores k row minima as anchors. Steps C–F operate on $m = k(n-1)$ gaps.

Algorithm 1 (global). Factors out only the *single* fully-symmetric element $\sum_{i,j} e_i^j$ (treating every entry identically), with coefficient = global minimum. The global maximum is also recorded (see note below). Steps C–F then operate on a single globally-sorted list of $m = nk - 1$ gaps.

The single global sort of Algorithm 1 means that prefix elements can contain Ejis from *any* row, whereas in Algorithm 2 every prefix element contains Ejis from exactly one row. This is why Algorithm 2 is conceptually cleaner: it respects the coordinate structure of the vectors.

Dimension count.

	Algorithm 1	Algorithm 2
Symmetric parts extracted	1 (global)	k (per-row)
Gaps m	$nk-1$	$k(n-1)=nk-k$
Anchor values stored	2 (max & min)	k (row minima)
Hash embedding dimension $2m+1$	$2nk-1$	$2nk-2k+1$
Total output size	$\mathbf{2nk+1}$	$\mathbf{2nk+1-k}$

Algorithm 2 saves k output dimensions because per-row symmetric extraction produces $k-1$ fewer gaps than global extraction (since each row contributes $n-1$ gaps, totalling $k(n-1)=nk-k$, versus the global $nk-1$ gaps from Algorithm 1), and the $(s+1)$ weighting scheme in Step D ensures that the hash dimension savings exceed the extra anchor overhead.

Note on Algorithm 1's global maximum. In a perfect implementation, only the global minimum is needed as an anchor (it fixes the absolute scale; all relative gaps are encoded in the hash sum). The global maximum is technically redundant. It is included in Algorithm 1 for implementation simplicity; a TODO comment in the source code acknowledges this.

4 Worked Example: Algorithm 2, $n=4$, $k=3$

$$M = \left\{ \begin{pmatrix}4\\2\\3\end{pmatrix}, \begin{pmatrix}-3\\5\\1\end{pmatrix}, \begin{pmatrix}8\\9\\2\end{pmatrix}, \begin{pmatrix}2\\7\\2\end{pmatrix} \right\} \quad (n=4 \text{ vectors in } \mathbb{R}^3)$$

Step B: Extract row minima

Row minima: $m_0=-3$ (row 0), $m_1=2$ (row 1), $m_2=1$ (row 2). These three scalars are stored as anchors. After subtracting:

$$\text{balance} = \left\{ \begin{pmatrix}7\\0\\2\end{pmatrix}, \begin{pmatrix}0\\3\\0\end{pmatrix}, \begin{pmatrix}11\\7\\1\end{pmatrix}, \begin{pmatrix}5\\5\\1\end{pmatrix} \right\}$$

(All non-negative.) The debug check in the source code verifies that $\sum_s \alpha_s \cdot (L_s/(s+1)) + \text{broadcast}(m_i)$ recovers the original M exactly.

Step C: Per-row Abel summation

For each row, sort vectors descending and compute gaps:

Row i	Sorted descent (value: vector j)	Gaps δ	Prefix elements $V_r^{(i)}$
0	$11(j{=}2),\ 7(j{=}0),\ 5(j{=}3),\ 0(j{=}1)$	4, 2, 5	$\{e_0^2\}$; $\{e_0^2, e_0^0\}$; $\{e_0^2, e_0^0, e_0^3\}$
1	$7(j{=}2),\ 5(j{=}3),\ 3(j{=}1),\ 0(j{=}0)$	2, 2, 3	$\{e_1^2\}$; $\{e_1^2, e_1^3\}$; $\{e_1^2, e_1^3, e_1^1\}$
2	$2(j{=}0),\ 1(j{=}2),\ 1(j{=}3),\ 0(j{=}1)$	1, 0, 1	$\{e_2^0\}$; $\{e_2^0, e_2^2\}$; $\{e_2^0, e_2^2, e_2^3\}$

At this stage in the code, each pair is stored as $(\delta, \texttt{MSV})$ where the MSV is an individual prefix set — it is a valid basis element but not yet in its final cumulative form.

Steps D: Merge, sort, and second Abel summation

Merge all $9 = k(n-1)$ pairs and sort by gap (i.e. δ) descending:

$$5,\ 4,\ 3,\ 2,\ 2,\ 2,\ 1,\ 1,\ 0$$

The table below assigns the rank index s, shows which prefix element $V_{(s)}$ each sorted gap is associated with (written as an Eji sum), and accumulates these into the cumulative LinComb $L_s = V_{(0)} + \cdots + V_{(s)}$. Terms within each L_s expression are grouped by coordinate row i.

			$\delta_{(s)} - \delta_{(s+1)}$	$(s+1)\Delta_{(s)}$	$L_{(s-1)} + V_{(s)}$
s	$\delta_{(s)}$	$V_{(s)}$	$\Delta_{(s)}$	α_s	$L_{(s)}$
0	5	$e_0^0 + e_0^2 + e_0^3$	1	1	$e_0^0 + e_0^2 + e_0^3$
1	4	e_0^2	1	2	$e_0^0 + 2e_0^2 + e_0^3$
2	3	$e_1^1 + e_1^2 + e_1^3$	1	3	$e_0^0 + 2e_0^2 + e_0^3$ $+ e_1^1 + e_1^2 + e_1^3$
3	2	$e_0^0 + e_0^2$	0	0	$2e_0^0 + 3e_0^2 + e_0^3$ $+ e_1^1 + e_1^2 + e_1^3$
4	2	e_1^2	0	0	$2e_0^0 + 3e_0^2 + e_0^3$ $+ e_1^1 + 2e_1^2 + e_1^3$
5	2	$e_1^2 + e_1^3$	1	6	$2e_0^0 + 3e_0^2 + e_0^3$ $+ e_1^1 + 3e_1^2 + 2e_1^3$
6	1	e_2^0	0	0	$2e_0^0 + 3e_0^2 + e_0^3$ $+ e_1^1 + 3e_1^2 + 2e_1^3$ $+ e_2^0$
7	1	$e_2^0 + e_2^2 + e_2^3$	1	8	$2e_0^0 + 3e_0^2 + e_0^3$ $+ e_1^1 + 3e_1^2 + 2e_1^3$ $+ 2e_2^0 + e_2^2 + e_2^3$
8	0	$e_2^0 + e_2^2$	0	0	$2e_0^0 + 3e_0^2 + e_0^3$ $+ e_1^1 + 3e_1^2 + 2e_1^3$ $+ 3e_2^0 + 2e_2^2 + e_2^3$

Coincidence in this example. Every non-zero Δ (i.e. every difference between successive δ values) happens to equal 1 here (e.g. $\delta_{(0)} - \delta_{(1)} = 5 - 4 = 1$, $\delta_{(5)} - \delta_{(6)} = 2 - 1 = 1$, etc.), so the non-zero α_s values equal $s{+}1$. This is a coincidence of the chosen data, not a general rule: $\alpha_s = (s{+}1) \times$ (gap difference), and the gap difference can be any non-negative integer.

The cumulative Eji_LinComb for each non-zero term. Count matrix: rows = coordinates $i{=}0,1,2$; columns = vectors $j{=}0,1,2,3$. The last column shows $L_s/(s{+}1)$ — the normalised basis element, i.e. the average of the first $s{+}1$ prefix MSVs. Terms of e_i^j with coefficient 0 are omitted; terms are grouped by coordinate row i.

s	α_s	L_s (count matrix, idx s+1)	Normalised basis element $L_s/(s+1)$
0	1	$\begin{pmatrix}1&0&1&1\\0&0&0&0\\0&0&0&0\end{pmatrix}$, idx 1	$e_0^0 + e_0^2 + e_0^3$
1	2	$\begin{pmatrix}1&0&2&1\\0&0&0&0\\0&0&0&0\end{pmatrix}$, idx 2	$\frac{1}{2}(e_0^0 + 2e_0^2 + e_0^3)$
2	3	$\begin{pmatrix}1&0&2&1\\0&1&1&1\\0&0&0&0\end{pmatrix}$, idx 3	$\frac{1}{3}(e_0^0 + 2e_0^2 + e_0^3 + e_1^1 + e_1^2 + e_1^3)$
5	6	$\begin{pmatrix}2&0&3&1\\0&1&3&2\\0&0&0&0\end{pmatrix}$, idx 6	$\frac{1}{6}(2e_0^0 + 3e_0^2 + e_0^3 + e_1^1 + 3e_1^2 + 2e_1^3)$
7	8	$\begin{pmatrix}2&0&3&1\\0&1&3&2\\2&0&1&1\end{pmatrix}$, idx 8	$\frac{1}{8}(2e_0^0 + 3e_0^2 + e_0^3 + e_1^1 + 3e_1^2 + 2e_1^3 + 2e_2^0 + e_2^2 + e_2^3)$

(Terms with $\alpha_s = 0$ contribute nothing and are omitted.)

Step E: Canonicalise

Sort the columns of each count matrix lexicographically (the column-sort permutation "forgets" which vector was which, achieving multiset invariance). After canonicalisation, L_s becomes $\widetilde{L_s}$.

Step F: Hash and assemble

Each $\widetilde{L_s}$ (along with its index) is hashed to a point $p_s \in [0,1]^{19}$ (since $N_{\text{hash}} = 2\times9+1 = 19$). The output is:

$$f(M) \;=\; \big[\, \underbrace{-3,\ 2,\ 1}_{\text{3 row min.}},\ 1\,p_0 + 2\,p_1 + 3\,p_2 + 6\,p_5 + 8\,p_7 \,\big] \;\in\; \mathbb{R}^{22}.$$

5 Worked Example: Algorithm 1, $n = 4$, $k = 2$

$$M \;=\; \left\{ \begin{pmatrix}4\\2\end{pmatrix}, \begin{pmatrix}-3\\5\end{pmatrix}, \begin{pmatrix}8\\9\end{pmatrix}, \begin{pmatrix}2\\7\end{pmatrix} \right\} \quad (n = 4 \text{ vectors in } \mathbb{R}^2)$$

Step B: Extract global minimum (and maximum)

Global min $= -3$, global max $= 9$. Both stored as anchors. Balance has all non-negative entries.

Steps C–D: Global sort and Abel summation

The balance after subtracting the global minimum -3 from every entry is:

$$\left\{ \begin{pmatrix}7\\5\end{pmatrix}, \begin{pmatrix}0\\8\end{pmatrix}, \begin{pmatrix}11\\12\end{pmatrix}, \begin{pmatrix}5\\10\end{pmatrix} \right\}$$

Sorted globally descending (value: label):

$$12(e_1^2),\ 11(e_0^2),\ 10(e_1^3),\ 8(e_1^1),\ 7(e_0^0),\ 5(e_1^0),\ 5(e_0^3),\ 0(e_0^1).$$

The 7 gaps are $1, 1, 2, 1, 2, 0, 5$. After global re-sort by gap descending and second Abel summation: $\alpha_0 = 3,\ \alpha_2 = 3,\ \alpha_5 = 6$ (others zero).

$N_{\text{hash}} = 2(8-1)+1 = 15$. Output:

$$f(M) \;=\; \big[\, 3\,p_0 + 3\,p_2 + 6\,p_5,\ 9,\ -3 \,\big] \;\in\; \mathbb{R}^{17},$$

numerically (from the code's debug output):

$$[\,7.07,\ 6.91,\ 2.78,\ 3.50,\ 1.01,\ 5.51,\ 4.07,\ 4.90,\ 10.02,\ 9.51,\ 5.13,\ 2.97,\ 8.04,\ 5.34,\ 7.41,\ 9,\ -3\,].$$

6 Output Size and the Whitney Embedding Theorem

Both algorithms produce output of size $\approx 2nk$, which is not overhead to be engineered away — it is intrinsic. The input space $SP^n(\mathbb{R}^k)$ is an nk-dimensional manifold. The **Whitney Embedding Theorem** states that any smooth d-dimensional manifold embeds smoothly into $\mathbb{R}^{2d}$, and $2d$ is tight in general. With output $\approx 2nk$, both algorithms sit right at the Whitney bound.

The factor of ≈ 2 is in fact the price of the cone-dimension mechanism in Step F: encoding m positive coefficients together with the identities of m basis elements inside a single point requires $\mathbb{R}^{2m+1}$. This same mechanism is what delivers C^0 continuity (shared faces at zero-coefficient transitions). The factor of two is the minimum price of a smooth embedding, and the algorithms pay almost exactly that.

Whether $2nk$ (or $2nk + 1$, or $2nk + 1 - k$) is optimal for the specific manifold $SP^n(\mathbb{R}^k)$, or whether the known structure of that space admits a tighter construction, remains open.